CHAPTER 1

Introduction

MapReduce [45] is a programming model for expressing distributed computations on massive amounts of data and an execution framework for large-scale data processing on clusters of commodity servers. It was originally developed by Google and built on well-known principles in parallel and distributed processing dating back several decades. MapReduce has since enjoyed widespread adoption via an open-source implementation called Hadoop, whose development was led by Yahoo (now an Apache project). Today, a vibrant software ecosystem has sprung up around Hadoop, with significant activity in both industry and academia.

This book is about scalable approaches to processing large amounts of text with MapReduce. Given this focus, it makes sense to start with the most basic question: Why? There are many answers to this question, but we focus on two. First, "big data" is a fact of the world, and therefore an issue that real-world systems must grapple with. Second, across a wide range of text processing applications, more data translates into more effective algorithms, and thus it makes sense to take advantage of the plentiful amounts of data that surround us.

Modern information societies are defined by vast repositories of data, both public and private. Therefore, any practical application must be able to scale up to datasets of interest. For many, this means scaling up to the web, or at least a non-trivial fraction thereof. Any organization built around gathering, analyzing, monitoring, filtering, searching, or organizing web content must tackle large-data problems: "web-scale" processing is practically synonymous with data-intensive processing. This observation applies not only to well-established internet companies, but also countless startups and niche players as well. Just think, how many companies do you know that start their pitch with "we're going to harvest information on the web and. . . "?

Another strong area of growth is the analysis of user behavior data. Any operator of a moderately successful website can record user activity and in a matter of weeks (or sooner) be drowning in a torrent of log data. In fact, logging user behavior generates so much data that many organizations simply can't cope with the volume, and either turn the functionality off or throw away data after some time. This represents lost opportunities, as there is a broadly-held belief that great value lies in insights derived from mining such data. Knowing what users look at, what they click on, how much time they spend on a web page, etc. leads to better business decisions and competitive advantages. Broadly, this is known as business intelligence, which encompasses a wide range of technologies including data warehousing, data mining, and analytics.

How much data are we talking about? A few examples: Google grew from processing 100 TB of data a day with MapReduce in 2004 [45] to processing 20 PB a day with MapReduce in 2008 [46]. In April 2009, a blog post[1] was written about eBay's two enormous data warehouses: one with 2 petabytes of user data, and the other with 6.5 petabytes of user data spanning 170 trillion records and growing by 150 billion new records per day. Shortly thereafter, Facebook revealed[2] similarly impressive numbers, boasting of 2.5 petabytes of user data, growing at about 15 terabytes per day. Petabyte datasets are rapidly becoming the norm, and the trends are clear: our ability to store data is fast overwhelming our ability to process what we store. More distressing, increases in capacity are outpacing improvements in bandwidth such that our ability to even *read* back what we store is deteriorating [91]. Disk capacities have grown from tens of megabytes in the mid-1980s to about a couple of terabytes today (several orders of magnitude). On the other hand, latency and bandwidth have improved relatively little: in the case of latency, perhaps 2× improvement during the last quarter century, and in the case of bandwidth, perhaps 50×. Given the tendency for individuals and organizations to continuously fill up whatever capacity is available, large-data problems are growing increasingly severe.

Moving beyond the commercial sphere, many have recognized the importance of data management in many scientific disciplines, where petabyte-scale datasets are also becoming increasingly common [21]. For example:

- The high-energy physics community was already describing experiences with petabyte-scale databases back in 2005 [20]. Today, the Large Hadron Collider (LHC) near Geneva is the world's largest particle accelerator, designed to probe the mysteries of the universe, including the fundamental nature of matter, by recreating conditions shortly following the Big Bang. When it becomes fully operational, the LHC will produce roughly 15 petabytes of data a year.[3]

- Astronomers have long recognized the importance of a "digital observatory" that would support the data needs of researchers across the globe—the Sloan Digital Sky Survey [145] is perhaps the most well known of these projects. Looking into the future, the Large Synoptic Survey Telescope (LSST) is a wide-field instrument that is capable of observing the entire sky every few days. When the telescope comes online around 2015 in Chile, its 3.2 gigapixel primary camera will produce approximately half a petabyte of archive images every month [19].

- The advent of next-generation DNA sequencing technology has created a deluge of sequence data that needs to be stored, organized, and delivered to scientists for

[1] http://www.dbms2.com/2009/04/30/ebays-two-enormous-data-warehouses/
[2] http://www.dbms2.com/2009/05/11/facebook-hadoop-and-hive/
[3] http://public.web.cern.ch/public/en/LHC/Computing-en.html

further study. Given the fundamental tenant in modern genetics that genotypes explain phenotypes, the impact of this technology is nothing less than transformative [103]. The European Bioinformatics Institute (EBI), which hosts a central repository of sequence data called EMBL-bank, has increased storage capacity from 2.5 petabytes in 2008 to 5 petabytes in 2009 [142]. Scientists are predicting that, in the not-so-distant future, sequencing an individual's genome will be no more complex than getting a blood test today—ushering a new era of personalized medicine, where interventions can be specifically targeted for an individual.

Increasingly, scientific breakthroughs will be powered by advanced computing capabilities that help researchers manipulate, explore, and mine massive datasets [72]—this has been hailed as the emerging "fourth paradigm" of science [73] (complementing theory, experiments, and simulations). In other areas of academia, particularly computer science, systems and algorithms incapable of scaling to massive real-world datasets run the danger of being dismissed as "toy systems" with limited utility. Large data is a fact of today's world and data-intensive processing is fast becoming a necessity, not merely a luxury or curiosity.

Although large data comes in a variety of forms, this book is primarily concerned with processing large amounts of text, but touches on other types of data as well (e.g., relational and graph data). The problems and solutions we discuss mostly fall into the disciplinary boundaries of natural language processing (NLP) and information retrieval (IR). Recent work in these fields is dominated by a data-driven, empirical approach, typically involving algorithms that attempt to capture statistical regularities in data for the purposes of some task or application. There are three components to this approach: data, representations of the data, and some method for capturing regularities in the data. Data are called *corpora* (singular, corpus) by NLP researchers and *collections* by those from the IR community. Aspects of the representations of the data are called *features*, which may be "superficial" and easy to extract, such as the words and sequences of words themselves, or "deep" and more difficult to extract, such as the grammatical relationship between words. Finally, algorithms or models are applied to capture regularities in the data in terms of the extracted features for some application. One common application, classification, is to sort text into categories. Examples include: Is this email spam or not spam? Is this word part of an address or a location? The first task is easy to understand, while the second task is an instance of what NLP researchers call named-entity detection [138], which is useful for local search and pinpointing locations on maps. Another common application is to rank texts according to some criteria—search is a good example, which involves ranking documents by relevance to the user's query. Another example is to automatically situate texts along a scale of "happiness", a task known as sentiment analysis or opinion mining [118], which has been applied to

everything from understanding political discourse in the blogosphere to predicting the movement of stock prices.

There is a growing body of evidence, at least in text processing, that of the three components discussed above (data, features, algorithms), data probably matters the most. Superficial word-level features coupled with simple models in most cases trump sophisticated models over deeper features and less data. But why can't we have our cake and eat it too? Why not both sophisticated models *and* deep features applied to lots of data? Because inference over sophisticated models and extraction of deep features are often computationally intensive, they don't scale well.

Consider a simple task such as determining the correct usage of easily confusable words such as "than" and "then" in English. One can view this as a supervised machine learning problem: we can train a classifier to disambiguate between the options, and then apply the classifier to new instances of the problem (say, as part of a grammar checker). Training data is fairly easy to come by—we can just gather a large corpus of texts and assume that most writers make correct choices (the training data may be noisy, since people make mistakes, but no matter). In 2001, Banko and Brill [14] published what has become a classic paper in natural language processing exploring the effects of training data size on classification accuracy, using this task as the specific example. They explored several classification algorithms (the exact ones aren't important, as we shall see), and not surprisingly, found that more data led to better accuracy. Across many different algorithms, the increase in accuracy was approximately linear in the log of the size of the training data. Furthermore, with increasing amounts of training data, the accuracy of different algorithms converged, such that pronounced differences in effectiveness observed on smaller datasets basically disappeared at scale. This led to a somewhat controversial conclusion (at least at the time): machine learning algorithms really don't matter, all that matters is the amount of data you have. This resulted in an even more controversial recommendation, delivered somewhat tongue-in-cheek: we should just give up working on algorithms and simply spend our time gathering data (while waiting for computers to become faster so we can process the data).

As another example, consider the problem of answering short, fact-based questions such as "Who shot Abraham Lincoln?" Instead of returning a list of documents that the user would then have to sort through, a question answering (QA) system would directly return the answer: John Wilkes Booth. This problem gained interest in the late 1990s, when natural language processing researchers approached the challenge with sophisticated linguistic processing techniques such as syntactic and semantic analysis. Around 2001, researchers discovered a far simpler approach to answering such questions based on pattern matching [27, 53, 92]. Suppose you wanted the answer to the above question. As it turns out, you can simply search for the phrase "shot Abraham Lincoln" on the web and look for what appears to its left. Or better yet, look through multiple instances

of this phrase and tally up the words that appear to the left. This simple strategy works surprisingly well, and has become known as the *redundancy-based approach* to question answering. It capitalizes on the insight that in a very large text collection (i.e., the web), answers to commonly-asked questions will be stated in obvious ways, such that pattern-matching techniques suffice to extract answers accurately.

Yet another example concerns smoothing in web-scale language models [25]. A language model is a probability distribution that characterizes the likelihood of observing a particular sequence of words, estimated from a large corpus of texts. They are useful in a variety of applications, such as speech recognition (to determine what the speaker is more likely to have said) and machine translation (to determine which of possible translations is the most fluent, as we will discuss in Section 6.4). Since there are infinitely many possible strings, and probabilities must be assigned to all of them, language modeling is a more challenging task than simply keeping track of which strings were seen how many times: some number of likely strings will never be encountered, even with lots and lots of training data! Most modern language models make the Markov assumption: in a n-gram language model, the conditional probability of a word is given by the $n-1$ previous words. Thus, by the chain rule, the probability of a sequence of words can be decomposed into the product of n-gram probabilities. Nevertheless, an enormous number of parameters must still be estimated from a training corpus: potentially V^n parameters, where V is the number of words in the vocabulary. Even if we treat every word on the web as the training corpus from which to estimate the n-gram probabilities, most n-grams—in any language, even English—will never have been seen. To cope with this sparseness, researchers have developed a number of smoothing techniques [35, 102, 79], which all share the basic idea of moving probability mass from observed to unseen events in a principled manner. Smoothing approaches vary in effectiveness, both in terms of intrinsic and application-specific metrics. In 2007, Brants et al. [25] described language models trained on up to two trillion words.[4] Their experiments compared a state-of-the-art approach known as Kneser-Ney smoothing [35] with another technique the authors affectionately referred to as "stupid backoff".[5] Not surprisingly, stupid backoff didn't work as well as Kneser-Ney smoothing on smaller corpora. However, it was simpler and could be trained on *more* data, which ultimately yielded better language models. That is, a simpler technique on more data beat a more sophisticated technique on less data.

[4] As an aside, it is interesting to observe the evolving definition of *large* over the years. Banko and Brill's paper in 2001 was titled *Scaling to Very Very Large Corpora for Natural Language Disambiguation*, and dealt with a corpus containing a billion words.

[5] As in, so stupid it couldn't possibly work.

Recently, three Google researchers summarized this data-driven philosophy in an essay titled *The Unreasonable Effectiveness of Data* [65].[6] Why is this so? It boils down to the fact that language *in the wild*, just like human behavior in general, is messy. Unlike, say, the interaction of subatomic particles, human *use* of language is not constrained by succinct, universal "laws of grammar". There are of course rules that govern the formation of words and sentences—for example, that verbs appear before objects in English, and that subjects and verbs must agree in number in many languages—but real-world language is affected by a multitude of other factors as well: people invent new words and phrases all the time, authors occasionally make mistakes, groups of individuals write within a shared context, etc. The Argentine writer Jorge Luis Borges wrote a famous allegorical one-paragraph story about a fictional society in which the art of cartography had gotten so advanced that their maps were as big as the lands they were describing.[7] The world, he would say, is the best description of itself. In the same way, the more observations we gather about language use, the more accurate a description we have of language itself. This, in turn, translates into more effective algorithms and systems.

So, in summary, why large data? In some ways, the first answer is similar to the reason people climb mountains: because they're there. But the second answer is even more compelling. Data represent the rising tide that lifts all boats—more data lead to better algorithms and systems for solving real-world problems. Now that we've addressed the *why*, let's tackle the *how*. Let's start with the obvious observation: data-intensive processing is beyond the capability of any individual machine and requires clusters—which means that large-data problems are fundamentally about organizing computations on dozens, hundreds, or even thousands of machines. This is exactly what MapReduce does, and the rest of this book is about the *how*.

1.1 COMPUTING IN THE CLOUDS

For better or for worse, it is often difficult to untangled MapReduce and large-data processing from the broader discourse on cloud computing. True, there is substantial promise in this new paradigm of computing, but unwarranted hype by the media and popular sources threatens its credibility in the long run. In some ways, cloud computing

[6] This title was inspired by a classic article titled *The Unreasonable Effectiveness of Mathematics in the Natural Sciences* [155]. This is somewhat ironic in that the original article lauded the beauty and elegance of mathematical models in capturing natural phenomena, which is the exact opposite of the data-driven approach.

[7] *On Exactitude in Science* [23]. A similar exchange appears in Chapter XI of *Sylvie and Bruno Concluded* by Lewis Carroll (1893).

is simply brilliant marketing. Before clouds, there were grids,[8] and before grids, there were vector supercomputers, each having claimed to be the best thing since sliced bread.

So what exactly is cloud computing? This is one of those questions where ten experts will give eleven different answers; in fact, countless papers have been written simply to attempt to define the term (e.g., [9, 31, 149], just to name a few examples). Here we offer up our own thoughts and attempt to explain how cloud computing relates to MapReduce and data-intensive processing.

At the most superficial level, everything that used to be called web applications has been rebranded to become "cloud applications", which includes what we have previously called "Web 2.0" sites. In fact, anything running inside a browser that gathers and stores user-generated content now qualifies as an example of cloud computing. This includes social-networking services such as Facebook, video-sharing sites such as YouTube, web-based email services such as Gmail, and applications such as Google Docs. In this context, the cloud simply refers to the servers that power these sites, and user data is said to reside "in the cloud". The accumulation of vast quantities of user data creates large-data problems, many of which are suitable for MapReduce. To give two concrete examples: a social-networking site analyzes connections in the enormous globe-spanning graph of friendships to recommend new connections. An online email service analyzes messages and user behavior to optimize ad selection and placement. These are all large-data problems that have been tackled with MapReduce.[9]

Another important facet of cloud computing is what's more precisely known as utility computing [129, 31]. As the name implies, the idea behind utility computing is to treat computing resource as a metered service, like electricity or natural gas. The idea harkens back to the days of time-sharing machines, and in truth isn't very different from this antiquated form of computing. Under this model, a "cloud user" can dynamically provision any amount of computing resources from a "cloud provider" on demand and only pay for what is consumed. In practical terms, the user is paying for access to virtual machine instances that run a standard operating system such as Linux. Virtualization technology (e.g., [15]) is used by the cloud provider to allocate available physical resources and enforce isolation between multiple users that may be sharing the

[8] What *is* the difference between cloud computing and grid computing? Although both tackle the fundamental problem of how best to bring computational resources to bear on large and difficult problems, they start with different assumptions. Whereas clouds are assumed to be relatively homogeneous servers that reside in a datacenter or are distributed across a relatively small number of datacenters controlled by a single organization, grids are assumed to be a less tightly-coupled federation of heterogeneous resources under the control of distinct but cooperative organizations. As a result, grid computing tends to deal with tasks that are coarser-grained, and must deal with the practicalities of a federated environment, e.g., verifying credentials across multiple administrative domains. Grid computing has adopted a middleware-based approach for tackling many of these challenges.

[9] The first example is Facebook, a well-known user of Hadoop, in exactly the manner as described [68]. The second is, of course, Google, which uses MapReduce to continuously improve existing algorithms and to devise new algorithms for ad selection and placement.

same hardware. Once one or more virtual machine instances have been provisioned, the user has full control over the resources and can use them for arbitrary computation. Virtual machines that are no longer needed are destroyed, thereby freeing up physical resources that can be redirected to other users. Resource consumption is measured in some equivalent of machine-hours and users are charged in increments thereof.

Both users and providers benefit in the utility computing model. Users are freed from upfront capital investments necessary to build datacenters and substantial reoccurring costs in maintaining them. They also gain the important property of elasticity—as demand for computing resources grow, for example, from an unpredicted spike in customers, more resources can be seamlessly allocated from the cloud without an interruption in service. As demand falls, provisioned resources can be released. Prior to the advent of utility computing, coping with unexpected spikes in demand was fraught with challenges: under-provision and run the risk of service interruptions, or over-provision and tie up precious capital in idle machines that are depreciating.

From the utility provider point of view, this business also makes sense because large datacenters benefit from economies of scale and can be run more efficiently than smaller datacenters. In the same way that insurance works by aggregating risk and redistributing it, utility providers aggregate the computing demands for a large number of users. Although demand may fluctuate significantly for each user, overall trends in aggregate demand should be smooth and predictable, which allows the cloud provider to adjust capacity over time with less risk of either offering too much (resulting in inefficient use of capital) or too little (resulting in unsatisfied customers). In the world of utility computing, Amazon Web Services currently leads the way and remains the dominant player, but a number of other cloud providers populate a market that is becoming increasingly crowded. Most systems are based on proprietary infrastructure, but there is at least one, Eucalyptus [111], that is available open source. Increased competition will benefit cloud users, but what direct relevance does this have for MapReduce? The connection is quite simple: processing large amounts of data with MapReduce requires access to clusters with sufficient capacity. However, not everyone with large-data problems can afford to purchase and maintain clusters. This is where utility computing comes in: clusters of sufficient size can be provisioned only when the need arises, and users pay only as much as is required to solve their problems. This lowers the barrier to entry for data-intensive processing and makes MapReduce much more accessible.

A generalization of the utility computing concept is "everything as a service", which is itself a new take on the age-old idea of outsourcing. A cloud provider offering customers access to virtual machine instances is said to be offering infrastructure as a service, or IaaS for short. However, this may be too low level for many users. Enter platform as a service (PaaS), which is a rebranding of what used to be called hosted services in the "pre-cloud" era. Platform is used generically to refer to any set of well-defined

services on top of which users can build applications, deploy content, etc. This class of services is best exemplified by Google App Engine, which provides the backend datastore and API for anyone to build highly-scalable web applications. Google maintains the infrastructure, freeing the user from having to backup, upgrade, patch, or otherwise maintain basic services such as the storage layer or the programming environment. At an even higher level, cloud providers can offer software as a service (SaaS), as exemplified by Salesforce, a leader in customer relationship management (CRM) software. Other examples include outsourcing an entire organization's email to a third party, which is commonplace today.

What does this proliferation of services have to do with MapReduce? No doubt that "everything as a service" is driven by desires for greater business efficiencies, but scale and elasticity play important roles as well. The cloud allows seamless expansion of operations without the need for careful planning and supports scales that may otherwise be difficult or cost-prohibitive for an organization to achieve. Cloud services, just like MapReduce, represents the search for an appropriate level of abstraction and beneficial divisions of labor. IaaS is an abstraction over raw physical hardware—an organization might lack the capital, expertise, or interest in running datacenters, and therefore pays a cloud provider to do so on its behalf. The argument applies similarly to PaaS and SaaS. In the same vein, the MapReduce programming model is a powerful abstraction that separates the *what* from the *how* of data-intensive processing.

1.2 BIG IDEAS

Tackling large-data problems requires a distinct approach that sometimes runs counter to traditional models of computing. In this section, we discuss a number of "big ideas" behind MapReduce. To be fair, all of these ideas have been discussed in the computer science literature for some time (some for decades), and MapReduce is certainly not the first to adopt these ideas. Nevertheless, the engineers at Google deserve tremendous credit for pulling these various threads together and demonstrating the power of these ideas on a scale previously unheard of.

Scale "out", not "up". For data-intensive workloads, a large number of commodity low-end servers (i.e., the scaling "out" approach) is preferred over a small number of high-end servers (i.e., the scaling "up" approach). The latter approach of purchasing symmetric multi-processing (SMP) machines with a large number of processor sockets (dozens, even hundreds) and a large amount of shared memory (hundreds or even thousands of gigabytes) is not cost effective, since the costs of such machines do not scale linearly (i.e., a machine with twice as many processors is often significantly more than twice as expensive). On the other hand, the low-end server market overlaps with the